

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

ТЕХНИЧЕСКОЕ ЗАДАНИЕ
по дисциплине Функциональное программирование

Система анализа статистики матчей НБА

Студент,
группы 5130201/40003

_____ Адиатуллин Т.Р

Преподаватель

_____ Моторин Д.Е.

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Содержание

1. Основание для выполнения работы	3
2. Исполнитель и соисполнители работы	3
3. Цели, задачи и исходные данные	3
3.1. Цель работы	3
3.2. Задачи	3
3.3. Исходные данные	4
4. Основное содержание работы	5
4.1. Разработка проекта	5
4.2. Отчётная документация	5
5. Основные требования к выполнению работы	6
5.1. Назначение программного комплекса	6
5.2. Состав программного комплекса	6
5.3. Технические характеристики	6
5.4. Функциональные требования	6
5.5. Требования к средствам разработки	8
6. Этапы и сроки выполнения работы	9
7. Результаты работы	10

1. Основание для выполнения работы

Основанием для выполнения работы является учебный план направления подготовки «Математика и компьютерные науки» кафедры ВШТИИ Института компьютерных наук и кибербезопасности Федерального государственного автономного образовательного учреждения высшего образования «Санкт-Петербургский политехнический университет Петра Великого». Работа выполняется в рамках дисциплины «Функциональное программирование», предусмотренной учебным планом четвертого семестра.

2. Исполнитель и соисполнители работы

Исполнитель: студент группы 5130201/40003, Адиатуллин Тимур Ринатович. Соисполнители: не предусмотрены, работа выполняется индивидуально.

3. Цели, задачи и исходные данные

3.1. Цель работы

Целью работы является разработка консольного приложения на языке Haskell для анализа статистики игроков и матчей Национальной баскетбольной ассоциации (НБА) за период 2010–2024 годов. Приложение обеспечивает загрузку данных из CSV-файлов, вычисление аналитических показателей эффективности игроков и команд, а также формирование текстовых отчётов.

3.2. Задачи

В процессе выполнения работы требуется решить следующие задачи:

1. Изучить предметную область спортивной аналитики НБА и форматы представления статистических данных.
2. Разработать техническое задание, определить структуру данных и архитектуру проекта.
3. Реализовать модуль чтения и валидации CSV-файлов со статистикой игроков и результатами матчей.
4. Реализовать математические функции: среднее, медиана, стандартное отклонение, рейтинг эффективности игрока (PER), коэффициент Пифагора для команд, нормализация показателей.
5. Разработать консольный интерфейс с меню навигации и фильтрацией данных по сезону, команде, игроку.

6. Реализовать систему логирования действий пользователя в файл с временными метками.
7. Обеспечить обработку ошибок ввода и некорректных данных.
8. Покрыть ключевые вычислительные функции тестами свойств с использованием QuickCheck.
9. Написать отчёт и защитить курсовой проект.

3.3. Исходные данные

Исходными данными для проведения работы являются CSV-файлы из открытого репозитория *NBA-Data-2010-2024* (автор: NocturneBear), доступного по адресу: <https://github.com/NocturneBear/NBA-Data-2010-2024>.

Датасет содержит данные о регулярных сезонах и плей-офф НБА за 2010–2024 годы и включает следующие файлы:

- `regular_season_box_scores_2010_2024_part_1/2/3.csv` — подробная статистика каждого игрока в каждом матче регулярного сезона: поля `season_year`, `game_date`, `gameId`, `teamName`, `personName`, `position`, `minutes`, `fieldGoalsMade`, `fieldGoalsAttempted`, `fieldGoalsPercentage`, `threePointersMade`, `threePointersPercentage`, `freeThrowsPercentage`, `reboundsTotal`, `assists`, `steals`, `blocks`, `turnovers`, `points`, `plusMinus`
- `play_off_box_scores_2010_2024.csv` — аналогичная статистика для матчей плей-офф;
- `regular_season_totals_2010_2024.csv` — агрегированная командная статистика за матч: поля `TEAM_NAME`, `GAME_DATE`, `MATCHUP`, `WL`, `PTS`, `FG_PCT`, `REB`, `AST`, `PLUS_MINUS` и другие;
- `play_off_totals_2010_2024.csv` — аналогичная командная статистика для плей-офф.

Дополнительным входным файлом является конфигурационный файл `config.json` задающий параметры сессии: выбранный сезон, тип турнира (регулярный сезон или плей-офф), фильтры по команде и минимальное количество сыгранных матчей.

4. Основное содержание работы

4.1. Разработка проекта

В рамках курсового проекта разрабатывается консольное приложение на языке Haskell с использованием системы сборки Stack. Проект организован в виде следующих программных модулей:

- `Parser` — чтение CSV-файлов (box scores и totals), разбор и валидация конфигурации JSON;
- `Types` — определение типов данных: `PlayerGame`, `TeamGame`, `Config` и других;
- `Stats` — математические функции: среднее, медиана, стандартное отклонение, PER, коэффициент Пифагора, нормализация;
- `Filter` — фильтрация и сортировка записей по параметрам пользователя;
- `Report` — формирование текстовых отчётов и экспорт результатов в файлы;
- `Logger` — запись действий пользователя в `session.log`;
- `UI` — консольное меню и обработка пользовательского ввода.

4.2. Отчётная документация

По результатам работы формируется отчёт, включающий настоящее техническое задание, описание архитектуры и диаграмму типов и модулей, описание технических особенностей реализации, примеры работы программы, руководство пользователя, список литературы (не менее 5 источников) и приложение с исходным кодом.

5. Основные требования к выполнению работы

5.1. Назначение программного комплекса

Разрабатываемый программный комплекс предназначен для аналитиков, тренеров и любителей баскетбола, желающих получать структурированную статистическую информацию по игрокам и командам НБА за период 2010–2024 годов, строить рейтинги и сравнивать показатели без использования сторонних веб-сервисов.

5.2. Состав программного комплекса

- исполняемый файл консольного приложения (собирается командой `stack build`);
- входные файлы данных: CSV-файлы датасета `NBA-Data-2010-2024` и `config.json`;
- выходные файлы: `report.txt` (аналитический отчёт), `top_players.csv` (экспорт рейтинга), `session.log` (лог действий).

5.3. Технические характеристики

- поддержка входных CSV-файлов объёмом до 200 000 строк (с учётом разбивки `box scores` на три части);
- время отклика на команду пользователя не более 5 секунд при загруженном датасете;
- корректная обработка кодировки UTF-8;
- работоспособность на ОС Linux, macOS, Windows (через Stack).

5.4. Функциональные требования

1. **Загрузка данных.** Программа читает и объединяет CSV-файлы датасета, валидирует типы и диапазоны значений, выводит информативные сообщения при ошибках парсинга.
2. **Топ игроков по показателю.** По выбранному числовому полю (`points`, `reboundsTotal`, `assists` и др.) за указанный сезон программа вычисляет среднее по каждому игроку и выводит ранжированный список. Среднее считается по формуле:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

Пример: топ-5 игроков по средним очкам за сезон 2023 из `regular_season_box_s`

3. **Профиль игрока по сезонам.** Для выбранного игрока программа агрегирует статистику по каждому сезону и дополнительно вычисляет упрощённый рейтинг эффективности (PER):

$$PER = \frac{PTS + REB + AST + STL + BLK - TOV - (FGA - FGM) - (FTA - FTM)}{MIN}$$

где все поля берутся из `box_scores`. Итоговый PER за сезон — среднее по всем матчам игрока.

4. **Сравнение двух игроков.** Программа нормализует показатели обоих игроков относительно всей лиги по формуле min-max и выводит их рядом:

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}} \in [0, 1]$$

Пример: сравнение Curry и Lillard по очкам, передачам, 3P% и PER за сезон 2022.

5. **Рейтинг команд по коэффициенту Пифагора.** По данным `regular_season_total` (поля `PTS` и `PLUS_MINUS`) программа вычисляет ожидаемый процент побед каждой команды:

$$W\% = \frac{PF^e}{PF^e + PA^e}, \quad e = 13,91$$

и сравнивает его с реальным. Разница указывает на «везение» или «невезение» команды в сезоне.

6. **Плей-офф против регулярного сезона.** Для выбранного игрока программа считает средние по `regular_season_box_scores` и `play_off_box_scores` отдельно и выводит дельту по каждому показателю:

$$\Delta = \bar{x}_{playoff} - \bar{x}_{regular}$$

Пример: Nikola Jokic, сезон 2023 — рост по очкам $\Delta = +5,7$, по подборам $\Delta = +1,3$.

7. **Статистическая сводка по позициям.** По выбранному полю программа группирует игроков по позиции (`position`) и для каждой группы вычисляет среднее, медиану и стандартное отклонение:

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2}$$

Пример: распределение очков по позициям PG / SG / SF / PF / C за сезон 2022.

8. **Лучшие матчи игрока.** Фильтрация строк `box_scores` по имени игрока с сортировкой по выбранному показателю и постраничным выводом: дата, соперник, статистика матча, результат (W/L).
9. **Экспорт результатов.** Сохранение отчёта по игроку или команде в `report.txt`; экспорт топ-листа в `top_players.csv`.
10. **Логирование.** Запись всех действий пользователя в `session.log` в формате `[YYYY-MM-DD HH:MM:SS] action: ...`
11. **Обработка ошибок.** Корректная обработка отсутствующих файлов, пустых полей, деления на ноль и неверного ввода без аварийного завершения программы.
12. **Тестирование свойств (QuickCheck):**
 - монотонность: сортировка по рейтингу даёт неубывающую последовательность;
 - корректность нормализации: все значения после нормализации лежат в `[0, 1]`;
 - симметрия среднего: перестановка элементов не меняет результат;
 - устойчивость к граничным случаям: пустой список, нулевые значения, все элементы одинаковы.

5.5. Требования к средствам разработки

При разработке программного обеспечения должны преимущественно использоваться стандартные средства и существующие библиотечные решения:

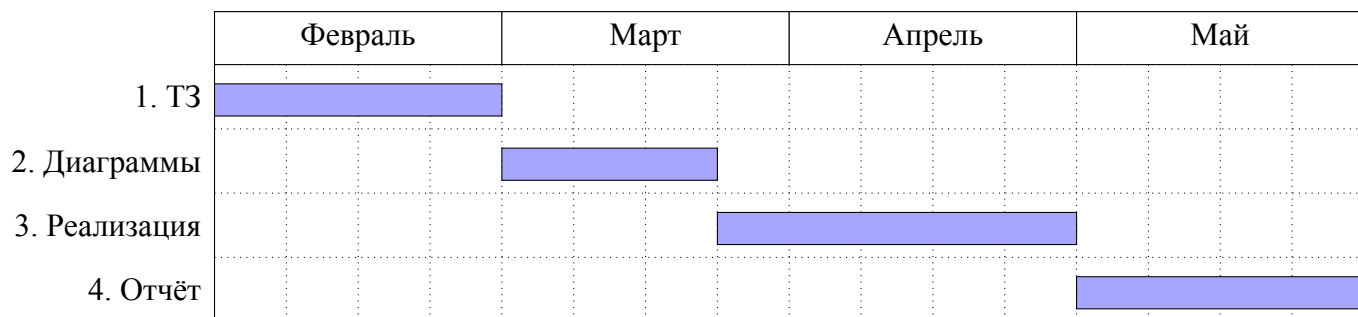
- язык программирования: Haskell (GHC $\geq$ 9.4);
- система сборки: Stack;
- `cassava` — парсинг CSV-файлов;
- `aeson` — чтение конфигурационного файла JSON;
- `time` — формирование временных меток в логах;
- `QuickCheck` — тестирование свойств математических функций.

6. Этапы и сроки выполнения работы

№	Этап	Начало	Окончание
1	Написание технического задания	01.02.2026	28.02.2026
2	Разработка диаграммы типов и модулей	01.03.2026	21.03.2026
3	Реализация программы на Haskell	22.03.2026	25.04.2026
4	Написание и защита отчёта	26.04.2026	24.05.2026

Таблица 1: Этапы выполнения курсового проекта

Диаграмма Ганта:



7. Результаты работы

1. **Этап 1.** Оформленное и подписанное техническое задание на курсовой проект.
2. **Этап 2.** Диаграмма типов данных и модульная схема проекта с описанием используемых библиотек и обоснованием их выбора.
3. **Этап 3.** Исходный код проекта на Haskell (Stack), размещённый в приватном репозитории GitHub с пошаговой историей коммитов и покрытием тестами QuickCheck.
4. **Этап 4.** Итоговый отчёт по курсовому проекту, защищённый перед преподавателем, с выгрузкой коммитов и фактической диаграммой Ганта.